

TinyOS-SUPPORTED HARDWARE

Modules	• Telos family • span • iMote2 • EPIC • iMote2 • Shimmer family • IRIS • Kmote, also called Telos Rev B • MicaZ • Mica2 • Mica2dot • NXTMOTe - TinyOS on LEGO MIND-STORMS NXT • Mülle • TMote Sky, also called Telos Rev B • TinyNode • Zolertia Z1 • UCMote Mini
MCU	• Texas Instruments MSP430 • Atmel - Atmega128 - Atmega128L - Atmega1281 • Intel px27ax • ARM Cortex M3 (under works)
Radio chips	• Texas Instruments/ChipCon - CC1000 - CC1100 (non-core group addition) - CC2420 - CC2500 (non-core group addition) • Infineon TDA5250 • Atmel - RF212 - RF230 • Semtech XE1205
Flash chips	• Atmel AT45DB • STMicroelectronics STM25P
Sensors	• Sensirion SHT11 - temperature and humidity sensor • Hamamatsu S1087 - visible light sensor • Hamamatsu S1087-01 - visible to infrared sensor • Intersema MS5534 - temperature and barometric pressure sensor • Taos TSL2550 - light sensor • Analog Devices ADXL202 - 2-axis accelerometer • LIS3L02DQ 3-axis accelerometer • MTS300/310 • AD5200, a digital potentiometer
Non-volatile storage	• Atmel AT45DB041 - 4Mbit SPI flash • Numonyx M25P80 - 8Mbit SPI flash (or STM25P80)
Other(s)	• Analog Devices ADG715 - Octal I2C-controlled analogue switch • Maxim (Dallas Semiconductor) DS2401 and DS2411 1-wire unique ID ROM • Dallas DS2745 I2C battery monitor • Dallas DS2782 I2C battery monitor

TinyOS supports

- Multi-hop
- Network-wide sub-millisecond time synchronisation through FTSP protocol
- Data collection to a designated root or gateway through CTP protocol
- Reliable data dissemination to every node in a network through Trickle algorithm
- Installing new binaries over the wireless network using Deluge
- Support for RPL, the forthcoming Internet standard for routing in low-power wireless networks, is currently underway

for low-power applications, Arduino is not. While Arduino focuses on sensing, digital I/O and computation, TinyOS supports these while adding functionalities for wireless as well. Since all IoT applications need some sort of wireless function, TinyOS fits in like a glove.

One crucial difference though is that, while Arduino is coded in C, TinyOS is written in nesC, a dialect of C developed specifically for TinyOS. All it does is add a few extra functionalities to aid programming.

Profile for TinyOS: the good and the not-so-good

If you are a newbie in the field of developing for the IoT, then you can rest easy knowing that the TinyOS environment not only supports multiple MCUs and radio chips, it is also extensively documented and checked. This has led to a robust and efficient code with very few bugs.

TinyOS's speciality lies in its strong support for low-power operation, especially wireless networking. These are extremely important when the OS uses single-hop and multi-hop communication. There is also extensive documentation and support for CC2420, the leading 802.15.4/ZigBee radio chip in use. Many of today's IoT standards are based on TinyOS designs and the countless hours put in by users worldwide.

Being low-power and low-resource-friendly does bring its share of issues too, though. TinyOS has, for a long time, suffered from a steep learning curve for its programming model, along with having issues in implementing processes that are computationally intensive. The thing is though that these two are highly correlated.

As TinyOS is intended for devices with very little RAM, its APIs are all non-blocking or split-phase. To understand non-blocking, let us take the example of a sensor sending a message. Because of limited RAM space, when TinyOS SEND()s a message, the send function is instructed to return immediately, even before the message is sent.

This might seem confusing but it is done to minimise the time taken by a single process. At a later stage, TinyOS calls a callback, sendDone(), to tell it that SEND() has been completed. The reason you cannot allow a process to run uninterruptedly is because then the OS cannot run other system processes and, in turn, might have to drop packets, timeouts and such.

A simple way around this is to break your multiple-step process into multiple single-step processes that run one after the other without interrupting each other. Granted, this may take some time to get used to, but the uptake is that there is enough and more documentation that will help you with the same.

Interestingly enough, the current version of TinyOS includes a thread library to help with cases of CPU-intensive applications by